

Python For Computational Problem Solving- UE26CS151A

Dr Manjula K
Professor, PCPS Anchor 2026
Department of CSE(AIML)

Introduction to JSON

What is Json

JSON = **J**ava**S**cript **O**bject **N**otation

It is a text-based format for representing structured data as key–value pairs, arrays, and nested combinations of both

Properties/Features:

- It's language independent
- The information is represented in dictionary form making retrieval easy
- It's lightweight as it stores info in text form
- It cannot hold executable data(including virus) making it safe to accept from outside sources

Example:

```
{
 "name": "Alison",
 "age": 21,
 "enrolled": true,
 "courses": ["PCPS", "DSA"]
}
```

It is used in Web APIs, config files etc

Introduction to JSON

Only **6** data types exist in JSON. Every one has a direct Python equivalent, which is why json.load() can hand you back native Python objects with zero manual conversion:

JSON type	Python type	Notes
object {}	dict	unordered key–value pairs
array []	list	ordered, can mix types
"string"	str	**must** use double quotes
number	int or float	JSON doesn't distinguish; Python infers from format
true / false	True / False	lowercase in JSON, capitalized in Python
null	None	represents "no value"

Syntax rules in JSON compared to Python

- **Keys must be double-quoted strings** — {'name': 'Adrian'} (Python dict syntax, single quotes) is not valid JSON
- **No trailing commas** — ["PCPS", "DSA",] will fail to parse. Python tuples/lists tolerate this, JSON does not
- **No comments allowed in JSON** — unlike Python # or JS //



- **Numbers can't have leading zeros** (007 is invalid) and can't be NaN/Infinity

Json module and functions

import json

Python's standard library

Core Json functions

There are 4 fundamental json functions used to serialize or deserialize a json file

	Serialize Python → Json	Deserialize Json → Python
File	json.dump(obj,file)	json.load(file)
String	json.dumps(obj) returns str	json.loads(str) → returns object

Json.dump()

- json.dump() serializes a Python object and writes the resulting JSON text directly into an open file.
- dump() returns None
- Because it streams the translated data straight to the disk, it is highly efficient
- It takes your Python data (like a dictionary or a list) and writes it directly into a .json file on your hard drive.

```
import json

 student = {"name": "Rigel", "age": 20,
"department": "CS"}

 with open("student.json", "w") as file:
    json.dump(student, file, indent=4)
```

dump() writes incrementally
If the program crashes mid-write without the file being properly closed, you can end up with a truncated, unparseable JSON file.
The with block guarantees the file closes (and flushes) even on error

Json.load()

- It reads a static .json file from your hard drive and ingests its contents directly into your Python environment.
- It deserializes json's universal text format into dynamic, manipulable Python objects like dictionaries(for objects) and lists(for arrays)
- It's input must be a file object opened in read mode('r').
- It automatically translates JSON structures to their Python equivalents without manual parsing

```
import json

 with open("data.json") as f:
     data=json.load(f)

print(data["name"])
print(type(data)) # <class 'dict'>
```

Json.dumps()

- It translates python object into json string format and stores it as a text variable within the system ram than writing it into disk.
- Because HTTP APIs and internet protocols communicate exclusively via text, dumps() is your primary mechanism for packaging complex backend data for network transmission.
- The letter ‘s’ here stands for string

```
import json

data = {"name": "Laken", "age": 21}

json_str = json.dumps(data,
indent=4)

print(json_str)
print(type(json_str))   # <class 'str'>
```

Json.loads()

- It deserializes JSON format strings within the program's into python objects.
- It's primary input is text variable within the program

- When your backend receives an API payload or a scraped web response, it arrives as a text string. loads() is the tool that decodes that raw text into actionable data you can manipulate.

- The letter 's' here stands for string

```
import json

 json_str = '{"name": "Holly", "age": 21}'

data = json.loads(json_str)

print(data["name"])
print(type(data)) # <class 'dict'>
```

- json_str above is a Python string containing double quotes inside single quotes.
- json is a strict data interchange protocol which mandates that strings must be in " "
- If you wrote '{'name': 'Asha'}' you'd get a Python syntax error

- **The Translation Crash:** JSON is very basic and only understands simple things like text, numbers, and lists. It has no idea what a Python date, set, or custom object is. If you try to save them directly, your code will crash. You must simplify them first: turn dates into text strings, and turn sets into normal lists.

- **The Sneaky Key Change**: Python lets you use numbers as dictionary keys (like {1: "apple"}). JSON absolutely refuses this and only allows text keys. It will quietly change your number 1 into a text string "1" without warning you. When you load your data back into Python, you might get bugs because your numbers are now text.

- **The Broken Data Trap**: Never blindly trust a JSON file from the internet or a user. If the file is missing even a single comma or quote, your entire program will instantly crash when it tries to read it. Always wrap your code in a try/except block so it catches the error instead of breaking your app.

- **Ugly Text Syndrome**: By default, JSON is terrified of accents and special characters. It will take a word like "café" and scramble it into ugly machine code like caf\u00e9. To force it to leave your text alone and keep it readable, you have to tell it ensure_ascii=False when you save the data.

- **The Windows vs. Mac Glitch**: Different computers assume different default text formats when opening files. If you do not specify a format, moving a JSON file between computers can scramble the text. Always explicitly write encoding="utf-8" inside your open() function to guarantee it works flawlessly everywhere.

Summary

What JSON is

- A text format for storing data using key-value pairs and lists. It looks a lot like a Python dictionary
- Used everywhere: web APIs, config files, saving/loading data
- Works the same across every programming language, which is why it's so popular for sharing data

JSON vs Python — matching types

- {} object → Python dict
- [] array → Python list
- "text" → str
- numbers → int / float
- true/false → True/False
- null → None

The 4 functions —

:"s" = string, no "s" = file

- json.dump(obj, file) — write Python data into a file
- json.load(file) — read a file back into Python
- json.dumps(obj) — turn Python data into a string
- json.loads(string) — turn a string back into Python data

Rules that trip people up

- JSON keys must always use double quotes — single quotes are not allowed
- No trailing commas after the last item
- No comments allowed inside JSON

Python For Computational Problem Solving

Time to be smart!

Read about TOON(data representation format born out of json) format

Project: The Campus Coffee Backend

The Mission: Build a simple Flask or FastAPI server that acts as a digital barista. It must receive a raw text coffee order from a student's phone, translate it into Python, calculate the bill, and send a text receipt back.

The Rules of the Game:

- The Internet is a Text Pipe: The phone app cannot send Python objects. It will send a raw JSON string.
- Your Code Only Speaks Python: You cannot do math on a text string. You must translate the incoming text into a live Python dictionary before calculating the price.
- The Return Trip: The phone app cannot read Python dictionaries. You must translate your final receipt back into a JSON string before sending it over the network.

Your Three Tasks:

1. Catch the Order (Deserialize): Create a web address (an endpoint) called /order. When the phone sends a JSON string like {"student": "John", "drink": "Latte", "quantity": 2}, your server must catch it and convert it into a Python dictionary.
2. Process the Data (Logic): Using Python, look at the dictionary. If a Latte costs $4, multiply that by the quantity to find the total price. Add a "status" tag that says "Order Confirmed".
3. Send the Receipt (Serialize): Create a new dictionary with the final details (student name, drink, total price, and status). Convert this live dictionary back into a flat JSON string and send it back to the phone.

The Test (How to know you passed):

- Input from Phone: {"student": "Alice", "drink": "Espresso", "quantity": 3}
- Required Output from Server: {"receipt_for": "Alice", "total_due": 12, "status": "Order Confirmed"}



Thank You

Department of CSE (AIML)
Dr Manjula K- manjulak@pes.edu

Ack: Teaching Assistant - Sudarshan G(CSE AIML)